💡 Esta es probablemente la lección más importante de **todo el Módulo 2.**

Protocolos

Todo en las redes (incluyendo la internet) funciona a base de `protocolos`.

Por ejemplo, `IP` es un protocolo (Internet Protocol), así como `DNS`.

Un `protocolo` es un conjunto de reglas y convenciones que los sistemas deben seguir para intercambiar datos y comunicarse.

Hay muchos `protocolos` distintos involucrados en la comunicación del internet.

Pero al igual que con los conceptos de la lección pasada, la mayoría no son tan importantes para nosotros los desarrolladores.

Uno de los `protocolos` más usados en internet, y **ESENCIAL de conocer para cualquier desarrollador**, es `HTTP`.

HTTP

`HTTP` o Hyper Text Transfer Protocol es un protocolo `cliente-servidor` de transferencia de `hyper-texto`.

Con `hyper-texto` nos referimos a texto capaz de ser convertido en una página web por nuestros navegadores (*veremos un poco más sobre esto en las próximas lecciones*).

Es el `protocolo` estándar para enviar `requests` y recibir `responses`.

Cuando vamos a www.google.com, estamos haciendo un `HTTP` `request` a los servidores de Google.

Nuestros navegadores web son clientes expertos en `HTTP` y lo usan cada segundo sin que nosotros lo sepamos.



También existe HTTPS, que en general es lo mismo que HTTP con la S de secure.
Encripta la información en ambos extremos para más seguridad.

Todos los `requests` y `responses` incluyen características e información que **debemos de conocer al dedillo** como desarrolladores web, y vamos a ver las más importantes.

Path (`requests`)

Cada vez que accesamos una url, sabemos que el contenido va a variar dependiendo de la `ruta` a la que accesemos.

No es lo mismo accesar a https://www.facebook.com/ que a https://www.facebook.com/marketplace/.

A pesar de que ambas páginas son parte de Facebook, son rutas distintas que apuntan a secciones distintas.

Esto es gracias a `HTTP`! Cada ruta en realidad apunta a un archivo o *función distinta dentro del código del servidor*.

Entonces cada ruta se maneja de manera distinta y devolverá un `response` distinto.

Parámetros (`requests`)

Los `requests` también pueden tener `parámetros`, justo como una función de código.

Y de hecho al crear servidores con código, estos `parámetros` se reciben como *parámetros de la función* que maneje el `request`.

Existen dos tipos de `parámetros`:

Path Parameters

Van directamente **dentro** de la `ruta`.

Suelen ser requeridos.

`https://www.facebook.com/marketplace/item/342534535/`

`https://us06web.zoom.us/j/82660107422/join`

`api.whathever.com/resources/person/5465/info`

Tambien puede (y suele) haber varios en una misma ruta.

`https://www.reddit.com/r/code/comments/1cggbbm/evaluate_an_arithmetic_operation_from_a_string/`

Query Parameters

Tambien llamados `GET Parameters`.

Van al final de la `ruta` con de un signo de interrogación seguido de su nombre, un = y su valor.

Suelen ser opcionales.

`https://www.youtube.com/watch?v=dQw4w9WgXcQ`

`https://www.google.com/search?q=Clojure`

Tambien puede (y suele) haber varias unidos por un &.

`https://www.hackerrank.com/domains/algorithms?filters=warmup&isFullScreen=true`

Métodos (`requests`)

Al igual que un objeto de OOP, `HTTP` tiene *`métodos`* (también llamados *verbos*) que podemos usar para enviar distintos tipos de `requests` a los servidores.

Los servidores realizan distintas acciones dependiendo del tipo de `request` que se reciba.

Una misma `ruta` puede hacer cosas **totalmente distintas** dependiendo del `método` que se use.

No es lo mismo hacer un `GET` a `my.api.com/friends` que un `POST` a `my.api.com/friends`.

Estos son los métodos más comúnmente utilizados, y los que componen el famoso `CRUD` (Create, Read, Update, Delete):

Método	Qué hace	Ejemplo
`GET`	Obtiene información.	Cada vez que visitamos una página web.
`POST`	Publica / crea información.	Cada vez que publicamos un comentario y damos un like.
`PUT`	Reemplaza información (actualiza completamente).	Cada vez que editamos una publicación.
`PATCH`	Modifica información (actualiza parcialmente).	Cada vez que editamos una sección especifica de nuestro perfil de Facebook.
`DELETE`	Borra información.	Cada vez que borramos una foto.

Puedes leer más sobre estos y otros métodos HTTP aca: https://developer.mozilla.org/en-US/docs/Web/HTTP/methods

Status Codes (`responses`)

Todos los `responses` incluyen un `status code` (*código de estado*) universal que resume el resultado de dicho `request`.

Le comunican al cliente si su solicitud fue exitosa, o si ocurrió algo inesperado a la hora de procesarla.

Por ejemplo, el famoso `404`, que significa que no existe el recurso o página a la que el cliente quiere accesar.

Estos códigos se dividen en grupos de centesimos (de 100 en 100), y cada centesimos tiene un significado específico.

Por ejemplo, los `status codes` del grupo `200` (*200, 201, 202, 203, etc*) indican que el `request` fue exitoso.

💡 Es muy importante que podamos identificar qué significa un código con sólo saber su primer dígito.

Rango	Descripción
`1XX`	Información
`2XX`	Éxito
`3XX`	Redirección
`4XX`	Error del cliente / `request`.
`5XX`	Error del servidor

Más allá de su rango, cada código específico significa algo específico.

Por ejemplo, `200` es éxito. `201` es éxito creando algo. `202` es éxito recibiendo el `request`, pero no necesariamente que se haya llevado a cabo aún, etc etc.

Lo mejor al iniciar es **enfocarse en aprender los grupos**. Los códigos específicos se pueden ir aprendiendo con la experiencia.

Puedes leer más sobre estos y otros status codes `HTTP` aca: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Body

El `body` o *cuerpo* es el contenido del `request` o `response`.

Del lado del cliente, se suele usar exclusivamente en `requests` que creen o actualicen información, como `POST`, `PUT` o `PATCH`.

Puede tener varios formatos, pero el estándar es usar `JSON`.

Por ejemplo, cuando creamos un comentario en Youtube, es muy probable que el `body` del `request` `POST` se vea algo así:

Copiar

```
{
    "userId": 45356,
    "videoUrl": "https://www.youtube.com/watch?v=dQw4w9WgXcQ",
    "commentContent": "Este video me cambió la vida, gracias!"
}
```

Headers

Los `headers` nos permiten enviar información extra en los `requests` y `responses`.

Ya sea sobre el cliente, sobre el servidor, o sobre el `request` / `response` como tal.

Authorization.

Se estructuran en `key-value` pairs.

Cada `header` tiene un nombre y un valor.

Justo como los elementos de un *diccionario* o un `JSON`.

Existen `headers` **universales** que casi cada `request` / `response` debe tener.

Por ejemplo, `Content-Type`, que especifica el formato del `body` (y su valor normalmente es `application/json`).

Cómo se ven?

Aquí un ejemplo de un request para una búsqueda en Google:

Aquí un ejemplo hipotético (simplificado) de un request para publicar un comentario en Youtube:

Repaso

Un `protocolo` es un conjunto de reglas y convenciones.

`HTTP` es el protocolo de transferencia de `hyper-texto` más utilizado en internet.

Los `requests` son el objeto enviado por el cliente hacia el servidor.

- Todos tienen una `ruta`.
- Algunos tienen `parámetros`.
- Todos usan un `método`.
- Algunos tienen un `body` (cuando son de `método` POST, PUT o PATCH).
- Todos tienen `headers`.

Los `responses` son el objeto devuelto por el servidor hacia el cliente.

- Todos tienen un `status code` que especifica qué pasó al procesar el `request`.
- Algunos tienen un `body`.
- Todos tienen `headers`.

Característica	Requests	Responses
Path	✅	×
Parameters	🔘	×
Method	✅	×
Body	🔘	🔘
Headers	✅	✅

🔘 algunas veces
✅ siempre
× nunca

Cómo practicar HTTP

Ya lo vamos a ver 😉